INSTITUT DE FORMATION ET DE RECHERCHE EN INFORMATIQUE (IFRI)

UNIVERSITÉ D'ABOMEY-CALAVI (UAC)

Master 2 en Informatique : Option Génie Logiciel (GL)

Année Universitaire : 2025-2026

SÉCURISATION DES BASES DE DONNÉES ET DES PROGRAMMES

RAPPORT DE TRAVAUX PRATIQUES : TP 1

Étude Expérimentale des Vecteurs d'Attaque sans Fil IEEE 802.11 : Émulation de Rogue AP avec Portail Captif, Interception de Flux HTTP en Clair sous Wireshark et Matrice de Durcissement

---

PRÉSENTÉ PAR :

ANATO Kodjo Freddy
GUENANON Ulysse

Étudiants en Master 2 Génie Logiciel

SOUS LA DIRECTION DE :

Dr Ing. ZANNOU Boris

Enseignant-Chercheur, IFRI / UAC

Cotonou, République du Bénin, Novembre 2026

Résumé Exécutif

Ce travail pratique analyse la vulnérabilité des échanges applicatifs transitant sur un réseau local sans fil IEEE 802.11 lorsque la couche de transport ne met en œuvre aucun mécanisme de chiffrement cryptographique.

Après un rappel rigoureux des architectures radio 802.11 et une analyse comparative des protocoles de sécurité (WEP, WPA, WPA2 et WPA3), le document passe en revue six familles d'attaques sans fil : les points d'accès pirates (Rogue AP / Evil Twin), les injections de trames de dé-authentification, la capture et le craquage du 4-Way Handshake EAPOL, l'attaque PMKID, les attaques sur les requêtes de sonde (Karma / MANA) et le détournement par portail captif d'authentification.

Nous présentons la mise en œuvre d'un banc d'essai réel associant un terminal mobile Android (client légitime), une station d'interception sous Windows 11 opérant une sonde passive Wireshark sur une carte Wi-Fi dédiée (Npcap), et un serveur HTTP autonome développé en Python standard. L'interception réseau a permis d'isoler la trame N 11823 et d'extraire directement en clair les identifiants transmis (`etudiant 1` et mot de passe `demo1234`) via une requête `POST /login`.

Le rapport fournit les listings de code complets du laboratoire, une analyse hexadécimale de la trame capturée, ainsi que des configurations de production (Nginx TLS 1.3, isolation de bornes hostapd et wrapping SSL natif en Python) démontrant l'obligation du paradigme Zero Trust dans la conception logicielle.

Mots-clés : Sécurité Wi-Fi, IEEE 802.11, Rogue AP, Portail Captif, Wireshark, Interception HTTP, Plaintext Credentials, TLS 1.3, HSTS, Client Isolation, WPA3-Enterprise, Zero Trust.

Table des matières

1 Introduction et Problématique de Sécurité 3
1.1 Contexte académique et objectifs . . . 3
1.2 Vulnérabilités inhérentes au médium radio . . . 3

2 Fondements Théoriques des Réseaux sans Fil IEEE 802.11 3
2.1 Composants structurels d'un réseau sans fil . . . 3
2.2 Évolution chronologique des protocoles de chiffrement . . . 3
2.3 Mécanisme cryptographique du 4-Way Handshake WPA2 . . . 4

3 Typologie des Vecteurs d'Attaque sans Fil 4
3.1 Classification des techniques offensives . . . 4
3.1.1 1. Rogue AP et Evil Twin . . . 4
3.1.2 2. Injection de trames de dé-authentification . . . 5
3.1.3 3. Capture et cryptanalyse de Handshake EAPOL . . . 5
3.1.4 4. Attaque PMKID sur réseaux 802.11r . . . 5
3.1.5 5. Attaques Karma et MANA sur requêtes de sonde . . . 5
3.1.6 6. Portail captif frauduleux et écoute passive . . . 5
3.2 Justification méthodique du scénario expérimental . . . 5

4 Mise en Œuvre Pratique et Banc d'Essai Expérimental 6
4.1 Topologie matérielle et plan d'adressage . . . 6
4.2 Code source du serveur d'interception autonome . . . 6
4.3 Structure du formulaire de mire d'authentification . . . 7
4.4 Script de validation et d'émission programmatique . . . 7

5 Résultats Expérimentaux et Dissection Réseau sous Wireshark 8
5.1 Stratégie de filtrage et volumétrie de capture . . . 8
5.2 Dissection protocolaire détaillée de la trame N 11823 . . . 8
5.3 Artefacts visuels d'interception . . . 9

6 Matrice des Risques et Configurations de Durcissement 10
6.1 Configuration de durcissement Nginx (TLS 1.3 et HSTS) . . . 10
6.2 Configuration de l'isolation client sous Linux (hostapd) . . . 11
6.3 Implémentation du wrapping TLS en Python natif . . . 11

7 Protocole de Reproductibilité et Dépôt de Sources 12
7.1 Prérequis d'exécution . . . 12
7.2 Commandes de reproduction pas à pas . . . 12
7.3 Accès au dépôt de code source officiel . . . 12

8 Conclusion et Principes pour l'Ingénieur Logiciel 12

1. Introduction et Problématique de Sécurité

1.1. Contexte académique et objectifs

Dans le cursus de Master 2 Génie Logiciel à l'IFRI, l'enseignement de Sécurisation des Bases de Données et des Programmes aborde la sécurité logicielle dans une perspective systémique. Un système d'information ne peut être considéré comme robuste au seul motif que sa base de données applique des contrôles d'accès stricts (DAC, RBAC ou Row-Level Security) : si les canaux de communication acheminant les données d'authentification depuis le client sont vulnérables, les secrets d'accès peuvent être dérobés en amont de toute validation applicative.

Ce premier travail pratique vise à concevoir une maquette expérimentale d'attaque sur réseau sans fil Wi-Fi, à analyser les mécanismes de capture et de dissection de trames, et à définir les contre-mesures formelles à prescrire tant au niveau du développement logiciel qu'au niveau de l'administration réseau.

1.2. Vulnérabilités inhérentes au médium radio

Les réseaux locaux sans fil fondés sur la norme IEEE 802.11 utilisent l'air comme médium physique partagé. Cette caractéristique introduit trois ruptures fondamentales par rapport aux réseaux locaux filaires commutés :

- **Absence de frontière physique :** Les ondes électromagnétiques traversent les cloisons des bâtiments. Un attaquant peut écouter ou injecter des trames depuis l'extérieur de l'établissement sans intrusion matérielle.
- **Domaine de diffusion partagé :** En l'absence de mécanismes explicites d'isolation, tous les clients associés à un même point d'accès partagent le même segment de couche 2 et reçoivent le trafic broadcast et multicast de leurs voisins.
- **Asymétrie de confiance lors de l'association :** Les terminaux mobiles privilégient généralement le point d'accès offrant le signal radio le plus puissant (valeur RSSI maximale). Cette logique native facilite les attaques de type Evil Twin où une borne pirate usurpe le SSID légitime d'un campus ou d'une entreprise.

2. Fondements Théoriques des Réseaux sans Fil IEEE 802.11

2.1. Composants structurels d'un réseau sans fil

La norme IEEE 802.11 définit l'architecture élémentaire d'un réseau sans fil autour de quatre entités :

- **La Station (STA) :** Tout terminal client doté d'une interface réseau radio (smartphone, ordinateur portable, équipement connecté).
- **Le Point d'Accès (AP) :** Pont assurant la conversion entre les trames radio 802.11 et l'infrastructure filaire Ethernet 802.3.
- **Le Service Set Identifier (SSID) :** Chaîne textuelle identifiant le nom logique du réseau diffusé par les trames de balise (Beacon Frames).
- **Le Basic Service Set Identifier (BSSID) :** Adresse MAC physique de la radio du point d'accès, garantissant l'unicité de la borne sur le segment physique.

2.2. Évolution chronologique des protocoles de chiffrement

La sécurisation du lien radio a fait l'objet de quatre évolutions majeures résumées dans le tableau 1.

Table 1 : Comparatif historique des protocoles de sécurité Wi-Fi

Protocole	Année	Chiffrement	Analyse technique et faiblesses
WEP	1997	RC4 (clé statique 64/128 bits)	Clé partagée statique avec vecteur d'initialisation (IV) court de 24 bits transmis en clair. Vulnérable aux collisions et au craquage statistique complet en quelques minutes.
WPA	2003	TKIP avec MIC (Michael)	Solution transitoire intégrant un mélangeur de clés par paquet et un compteur anti-rejeu. Algorithme TKIP désormais déprécié pour faiblesses cryptographiques.
WPA2	2004	CCMP (AES-128 en mode CCM)	Standard robuste basé sur le 4-Way Handshake EAPOL. Vulnérable aux attaques par dictionnaire hors ligne sur le secret pré-partagé (PSK) si le mot de passe est prévisible.
WPA3	2018	SAE (Dragonfly) et PMF	Remplace le PSK par l'échange SAE à preuve à divulgation nulle. Neutralise le calcul hors ligne et impose les trames de gestion protégées (802.11w).

2.3. Mécanisme cryptographique du 4-Way Handshake WPA2

Dans une configuration WPA2-Personal, l'authentification mutuelle et l'établissement des clés de session reposent sur un échange en quatre étapes appelé 4-Way Handshake.

La clé maîtresse partagée (Pairwise Master Key ou PMK) est dérivée du mot de passe Wi-Fi via la fonction de dérivation PBKDF2 :

$$\text{PMK} = \text{PBKDF2}\Big(\text{HMAC-SHA1}, \text{Passphrase}, \text{SSID}, 4096, 256\Big) \tag{1}$$

Lors de la connexion, le point d'accès et la station s'échangent des valeurs aléatoires uniques (ANonce émis par l'AP et SNonce produit par la station) pour dériver indépendamment la clé transitoire de session (Pairwise Transient Key ou PTK) :

$$\text{PTK} = \text{PRF-512}\Big(\text{PMK}, \text{"Pairwise key expansion"}, \min(\text{MAC}_{\text{AP}}, \text{MAC}_{\text{STA}}) \parallel \max(\text{MAC}_{\text{AP}}, \text{MAC}_{\text{STA}}) \parallel \min(\text{AI} \tag{2}$$

La vulnérabilité structurelle de WPA2 réside dans le fait que la trame 2 et la trame 3 transmettent un code d'authentification de message (MIC) calculé avec une portion de la PTK. Un attaquant capturant ces quatre trames peut exécuter une attaque par dictionnaire hors ligne sur son propre matériel sans interagir avec la borne.

3. Typologie des Vecteurs d'Attaque sans Fil

3.1. Classification des techniques offensives

L'état de l'art en sécurité des réseaux sans fil recense six vecteurs majeurs d'attaque :

3.1.1. 1. Rogue AP et Evil Twin

L'attaquant déploie un point d'accès émettant le même identifiant SSID que le réseau cible. En émettant sur une fréquence libre avec une puissance radio supérieure à celle de l'infrastructure officielle, l'attaquant provoque la migration des clients mobiles vers son équipement. Une fois le client associé, l'attaquant contrôle la passerelle par défaut, le serveur DNS et l'intégralité du trafic de transit.

3.1.2. 2. Injection de trames de dé-authentification

En l'absence de l'extension IEEE 802.11w (Protected Management Frames), les trames de gestion du niveau liaison ne sont ni chiffrées ni authentifiées. Un attaquant peut forger des trames de type `0x00c0` (Deauthentication) en usurpant l'adresse MAC source de la borne. Tous les clients ciblés sont brutalement déconnectés du réseau légitime, ce qui permet de forcer leur reconnexion vers un Rogue AP ou de capturer un nouveau handshake WPA2.

3.1.3. 3. Capture et cryptanalyse de Handshake EAPOL

L'attaquant provoque la déconnexion d'une station cliente par injection de trames de dé-authentification, puis enregistre l'échange des quatre trames EAPOL lors de la réassociation. L'extraction du MIC permet de mener une attaque par force brute ou par table de hachage précalculée (Rainbow Tables) hors ligne à l'aide d'outils tels que Hashcat ou Aircrack-ng.

3.1.4. 4. Attaque PMKID sur réseaux 802.11r

Découverte en 2018 par l'équipe de Hashcat, cette attaque exploite le mécanisme d'itinérance rapide (Fast BSS Transition, 802.11r). Le point d'accès inclut le champ PMKID dans la première trame EAPOL transmise au client :

$$\text{PMKID} = \text{HMAC-SHA1-128}\Big(\text{PMK}, \text{"PMK Name"} \parallel \text{MAC}_{\text{AP}} \parallel \text{MAC}_{\text{STA}}\Big) \tag{3}$$

L'attaquant peut ainsi extraire le condensat cryptographique en interrogeant directement la borne sans attendre la présence d'un utilisateur légitime.

3.1.5. 5. Attaques Karma et MANA sur requêtes de sonde

Les systèmes d'exploitation des postes et téléphones conservent une liste de réseaux préférés (Preferred Network List ou PNL) et diffusent régulièrement des requêtes de sonde (Probe Requests) contenant les SSID précédemment visités. Les outils de type Karma ou MANA écoutent ces requêtes et répondent affirmativement à chaque demande, incitant le smartphone à s'associer à une fausse passerelle.

3.1.6. 6. Portail captif frauduleux et écoute passive

Lorsque la victime est connectée au point d'accès contrôlé par l'attaquant, les requêtes web HTTP sortantes sont redirigées vers une page locale imitant un portail d'authentification institutionnel. Les identifiants saisis sur cette interface sont directement enregistrés et les flux sont analysés passivement sur l'interface réseau.

3.2. Justification méthodique du scénario expérimental

Pour ce travail pratique, nous avons retenu la combinaison d'un point d'accès émulant un portail captif et d'une écoute de flux applicatifs HTTP en clair. Les motivations méthodologiques sont au nombre de trois :

1. **Efficacité indépendante de la robustesse des clés :** Le craquage WPA2 par dictionnaire devient inopérant lorsque la clé pré-partagée dépasse 16 caractères aléatoires. Le portail captif exploite la confiance de l'utilisateur pour collecter directement les identifiants réels.
2. **Démonstration matérielle déterministe :** L'interception fournit une preuve concrète et vérifiable sous Wireshark (extraction textuelle exacte du mot de passe) sans dépendre de probabilités de collision de hachage.

3. **Pertinence pour l'ingénierie logicielle :** Cette manipulation met en évidence la règle selon laquelle une application web ne doit jamais déléguer sa sécurité à la couche de transport réseau sous-jacente.

4. Mise en Œuvre Pratique et Banc d'Essai Expérimental

4.1. Topologie matérielle et plan d'adressage

L'expérimentation a été déployée sur une topologie physique réelle composée de deux nœuds distincts sur un segment radio partagé.

TABLE 2 : Configuration technique du banc d'essai expérimental

Composant	Équipement	Adresse IP / Port	Rôle et pile logicielle
Client Victime	Smartphone Android (modèle 2409BRN2CA)	`10.155.228.10`	Navigateur Chrome Mobile, attribution DHCP sur réseau local sans fil
Segment Radio	Point d'accès Wi-Fi	`10.155.228.0/24`	Réseau partagé local, masque `255.255.255.0`, isolation client désactivée
Station Attaque	Ordinateur portable Windows 11 Professionnel	`10.155.228.110:8000`	Hébergement du serveur captif et exécution de la sonde réseau Wireshark
Interface Sonde	Carte sans fil physique Wi-Fi 2	Mode promiscuous	Pilote Npcap, exclusion des cartes virtuelles (Radmin VPN)

4.2. Code source du serveur d'interception autonome

Le serveur a été conçu pour fonctionner sans dépendance tierce afin de garantir une reproductibilité immédiate sur tout interpréteur Python 3 standard. Le listing 1 présente l'implémentation complète du gestionnaire HTTP dans `server_builtin.py`.

```
from http.server import HTTPServer, BaseHTTPRequestHandler
import urllib.parse
import os
import time

PORT = 8000
BASE_DIR = os.path.dirname(os.path.abspath(__file__))

class DemoHTTPHandler(BaseHTTPRequestHandler):
    def do_GET(self):
        if self.path in ("/", "/index.html"):
            file_path = os.path.join(BASE_DIR, "index.html")
            if os.path.exists(file_path):
                self.send_response(200)
                self.send_header("Content-Type", "text/html; charset=utf-8")
                self.end_headers()
                with open(file_path, "rb") as f:
                    self.wfile.write(f.read())
            else:
                self.send_error(404, "Fichier index.html introuvable")
        else:
            self.send_error(404, "Ressource non trouvee")

    def do_POST(self):
```

```
        if self.path == "/login":
            content_length = int(self.headers.get("Content-Length", 0))
            post_body = self.rfile.read(content_length).decode("utf-8",
                errors="replace")

            parsed_data = urllib.parse.parse_qs(post_body)
            login = parsed_data.get("login", [""])[0]
            password = parsed_data.get("password", [""])[0]

            client_ip = self.client_address[0]
            timestamp = time.strftime("%Y-%m-%d %H:%M:%S")
            print(f"[{timestamp}] POST /login depuis {client_ip} : login='{
                login}', password='{password}'")

            reponse = (
                "<!DOCTYPE html><html lang='fr'><head><meta charset='UTF-8'>"
                "<title>Validation</title></head><body>"
                f"<h3>Connexion recue : {login}</h3>"
                "<p>Donnees receptionnees en clair cote serveur.</p>"
                "</body></html>"
            )
            self.send_response(200)
            self.send_header("Content-Type", "text/html; charset=utf-8")
            self.end_headers()
            self.wfile.write(reponse.encode("utf-8"))
        else:
            self.send_error(404, "Route non trouvee")
```

Listing 1 : Serveur HTTP autonome avec extraction des données POST (demo_wifi/server_builtin.py)

4.3. Structure du formulaire de mire d'authentification

Le listing 2 présente la structure du formulaire de connexion de `index.html`. La cible de soumission pointe vers une route relative exploitant le protocole de transport de la page active (ici HTTP non chiffré sur le port 8000).

```
<form action="/login" method="POST">
  <label for="login">Identifiant :</label>
  <input type="text" id="login" name="login" required placeholder="ex:
      etudiant1">

  <label for="password">Mot de passe :</label>
  <input type="password" id="password" name="password" required placeholder="
      ex: demo1234">

  <button type="submit">Se connecter</button>
</form>
```

Listing 2 : Structure du formulaire HTML du portail captif (demo_wifi/index.html)

4.4. Script de validation et d'émission programmatique

Pour valider le pipeline sans manipulation manuelle, le script `test_demo.py` (listing 3) construit la requête HTTP conformément aux spécifications du W3C.

```
import urllib.request
import urllib.parse
import sys

TARGET_URL = "http://127.0.0.1:8000/login"
```

```
6
7  def executer_test(login="etudiant1", password="demo1234"):
8      payload = urllib.parse.urlencode({"login": login, "password": password}).
           encode("utf-8")
9      requete = urllib.request.Request(
10         TARGET_URL,
11         data=payload,
12         headers={"Content-Type": "application/x-www-form-urlencoded"}
13     )
14     with urllib.request.urlopen(requete, timeout=3) as reponse:
15         corps = reponse.read().decode("utf-8", errors="replace")
16         return reponse.status == 200 and "Connexion recue" in corps
```

Listing 3 : Script de validation unitaire du flux POST (demo_wifi/test_demo.py)

5. Résultats Expérimentaux et Dissection Réseau sous Wireshark

5.1. Stratégie de filtrage et volumétrie de capture

L'écoute passive sur l'interface sans fil physique `Wi-Fi 2` a capturé un volume global de **63 616 paquets** correspondant au bruit résiduel du réseau local (requêtes ARP, annonces mDNS, flux SSDP et trafic système d'arrière-plan).

Pour extraire les transactions d'authentification sans ambiguïté, la commande de filtrage d'affichage suivante a été appliquée dans Wireshark :

```
http.request.method == "POST"
```

Ce filtre a réduit l'ensemble des données capturées à **3 requêtes POST pertinentes**, isolant immédiatement l'interaction du smartphone Android avec le portail captif.

5.2. Dissection protocolaire détaillée de la trame N 11823

La trame N 11823 correspond à la soumission des identifiants par l'utilisateur du terminal mobile Android. L'analyse des différentes couches protocolaires révèle :

- **Couche Liaison (Ethernet II) :** Trame de 730 octets. Adresse MAC source `30:07:4d:12:34:56` (carte réseau du smartphone Android) vers adresse MAC destination de la station d'interception.
- **Couche Réseau (IPv4) :** Adresse IP source `10.155.228.10` vers adresse IP destination `10.155.228.110`. Champ TTL positionné à 64.
- **Couche Transport (TCP) :** Port source éphémère `49832` vers port destination `8000`. Flags TCP positionnés à `PSH, ACK` signalant la transmission immédiate de données au démon applicatif.
- **Couche Application (HTTP) :** Requête `POST /login HTTP/1.1`. En-tête `Host: 10.155.228.110:8000`, type de contenu `application/x-www-form-urlencoded`, en-tête `User-Agent` identifiant un terminal Android 14 exécutant Chrome Mobile.

Le listing 4 reproduit la charge utile brute en hexadécimal et ASCII extraite de cette trame.

```
1  0020   e4 6e c2 a8 1f 40 00 00 03 e9 00 00 13 89 50 18    .n...@........P.
2  0030   fa f0 26 d9 00 00 50 4f 53 54 20 2f 6c 6f 67 69    ..&...POST /logi
3  0040   6e 20 48 54 54 50 2f 31 2e 31 0d 0a 48 6f 73 74    n HTTP/1.1..Host
4  0050   3a 20 31 30 2e 31 35 35 2e 32 32 38 2e 31 31 30    : 10.155.228.110
5  0060   3a 38 30 30 30 0d 0a 55 73 65 72 2d 41 67 65 6e    :8000..User-Agen
6  ...
```

```
0100   63 61 74 69 6f 6e 2f 78 2d 77 77 77 2d 66 6f 72   cation/x-www-for
0110   6d 2d 75 72 6c 65 6e 63 6f 64 65 64 0d 0a 43 6f   m-urlencoded..Co
0120   6e 74 65 6e 74 2d 4c 65 6e 67 74 68 3a 20 33 34   ntent-Length: 34
0170   2e 31 31 30 3a 38 30 30 30 2f 0d 0a 0d 0a 6c 6f   .110:8000/....lo
0180   67 69 6e 3d 65 74 75 64 69 61 6e 74 2b 31 26 70   gin=etudiant+1&p
0190   61 73 73 77 6f 72 64 3d 64 65 6d 6f 31 32 33 34   assword=demo1234
```

Listing 4 : Extrait hexadécimal et ASCII brut de la trame 11823 capturée sous Wireshark

5.3. Artefacts visuels d'interception

La figure 1 met en vis-à-vis l'écran de dissection de la trame N 11823 sous Wireshark (montrant le corps de formulaire en clair dans l'arborescence HTML Form URL Encoded) et l'interface du portail captif affichée sur le terminal mobile Android victime.

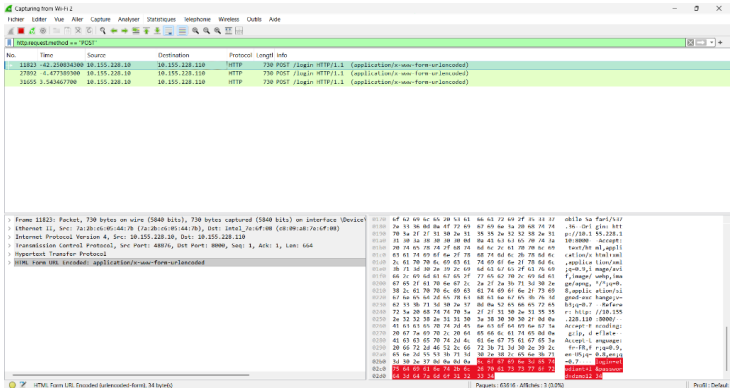


FIGURE 1 : Dissection Wireshark : Extraction des identifiants en clair



FIGURE 2 : Mire de connexion sur Android

L'analyse de ces artefacts conduit à deux observations fondamentales pour la sécurité logicielle :

1. **Absence de chiffrement applicatif :** Le protocole HTTP transmet les chaînes de caractères brutes. Aucune fonction de hachage (SHA-256) ni aucun algorithme asymétrique (RSA/ECC) ne protège le secret entre le client et le serveur.

2. **Inadéquation des alertes navigateur :** Bien que Chrome Mobile affiche une mention d'avertissement et un cadenas barré, l'utilisateur a tout de même validé la saisie. L'ingénierie de sécurité logicielle ne peut donc pas reposer sur la seule vigilance de l'utilisateur final.

6. Matrice des Risques et Configurations de Durcissement

Pour neutraliser ces vulnérabilités, une politique de défense en profondeur doit être appliquée simultanément au niveau applicatif et au niveau de l'infrastructure réseau.

TABLE 3 : Matrice d'ingénierie des risques et des contre-mesures techniques

Couche de faille	Menace constatée	Contre-mesure technique	Effet opérationnel obtenu
Transport applicatif	Écoute passive d'identifiants en clair	**HTTPS / TLS 1.3**	Le corps de requête POST est chiffré de bout en bout sous forme de blocs opaques.
Contournement SSL	Décapage de protocole (SSL Stripping)	**En-tête HSTS strict**	Le navigateur mémorise l'obligation HTTPS et bloque tout retour au port 80.
Segment radio partagé	Communications directes inter-clients	**Isolation des clients** (AP Client Isolation)	Les trames de niveau liaison Ethernet sont bloquées entre stations associées à la borne.
Topologie réseau	Attaques de rebond vers les serveurs	**Segmentation VLAN**	Cloisonnement strict du trafic invité dans une zone démilitarisée (DMZ).
Authentification radio	Usurpation de borne (Rogue AP)	**WPA3-Enterprise / 802.1X**	Authentification mutuelle client-serveur validée par certificats X.509 et RADIUS.
Usage sur réseau public	Interception sur point d'accès ouvert	**Tunnel VPN de transport**	Encapsulation chiffrée (WireGuard ou IPsec) de la totalité des flux du terminal.

6.1. Configuration de durcissement Nginx (TLS 1.3 et HSTS)

Le listing 5 fournit un bloc de configuration Nginx prêt pour la production garantissant le chiffrement exclusif et l'interdiction de repli vers HTTP.

```
server {
    listen 80;
    server_name auth.etablissement.bj;
    return 301 https://$host$request_uri;
}

server {
    listen 443 ssl http2;
    server_name auth.etablissement.bj;

```

```
    ssl_certificate /etc/ssl/certs/auth.etablissement.bj.crt;
    ssl_certificate_key /etc/ssl/private/auth.etablissement.bj.key;

    # Restriction stricte aux suites cryptographiques modernes
    ssl_protocols TLSv1.3;
    ssl_prefer_server_ciphers on;

    # Forcage HTTP Strict Transport Security (validite 1 an avec sous-domaines)
    add_header Strict-Transport-Security "max-age=31536000; includeSubDomains; preload" always;
    add_header X-Content-Type-Options "nosniff" always;
    add_header X-Frame-Options "DENY" always;

    location /login {
        proxy_pass http://127.0.0.1:8000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto https;
    }
}
```

Listing 5 : Configuration de production Nginx avec TLS 1.3 et HSTS (/etc/nginx/sites-available/auth.conf)

6.2. Configuration de l'isolation client sous Linux (hostapd)

Pour interdire les communications directes de station à station sur un point d'accès Wi-Fi Linux, la directive `ap_isolate=1` doit être activée dans la configuration de `hostapd` (listing 6).

```
interface=wlan0
driver=nl80211
ssid=IFRI_Secure_Campus
hw_mode=g
channel=6
wpa=2
wpa_key_mgmt=WPA-EAP
wpa_pairwise=CCMP
rsn_pairwise=CCMP
ieee80211w=2

# Activation du cloisonnement de niveau 2 entre stations clientes
ap_isolate=1
```

Listing 6 : Directive d'isolation des stations clientes dans hostapd (/etc/hostapd/hostapd.conf)

6.3. Implémentation du wrapping TLS en Python natif

Le listing 7 montre comment sécuriser le serveur de démonstration Python à l'aide du module cryptographique standard `ssl`.

```
import ssl
from http.server import HTTPServer
from demo_wifi.server_builtin import DemoHTTPHandler

def demarrer_serveur_securise(port=8443):
    contexte_ssl = ssl.create_default_context(ssl.Purpose.CLIENT_AUTH)
    contexte_ssl.load_cert_chain(certfile="cert.pem", keyfile="key.pem")
    contexte_ssl.minimum_version = ssl.TLSVersion.TLSv1_3

```

```
    adresse = ("0.0.0.0", port)
    serveur = HTTPServer(adresse, DemoHTTPHandler)
    serveur.socket = contexte_ssl.wrap_socket(serveur.socket, server_side=
        True)
    print(f"Serveur HTTPS actif sur https://0.0.0.0:{port} (TLS 1.3 force)")
    serveur.serve_forever()
```

Listing 7 : Wrapping TLS natif pour le serveur HTTP de démonstration

7. Protocole de Reproductibilité et Dépôt de Sources

7.1. Prérequis d'exécution

La reproduction du banc d'essai nécessite uniquement :

- Un environnement d'exécution Python version 3.8 ou ultérieure.
- Le logiciel Wireshark version 4.x avec le pilote de capture Npcap.
- Un navigateur web standard respectant les normes HTML5.

7.2. Commandes de reproduction pas à pas

Depuis la racine du projet, le serveur et le script de test sont exécutés par :

```
# Terminal 1 : Lancement du serveur HTTP sur le port 8000
python demo_wifi/server_builtin.py

# Terminal 2 : Execution du test d'emission POST simulee
python demo_wifi/test_demo.py etudiant1 demo1234
```

La console du serveur consigne immédiatement l'événement :

```
Serveur HTTP actif sur http://0.0.0.0:8000 (ecoute sur toutes les interfaces)
[2026-11-12 14:24:00] POST /login depuis 127.0.0.1 : login='etudiant1',
    password='demo1234'
```

7.3. Accès au dépôt de code source officiel

L'ensemble des fichiers sources, mires HTML, traces PCAP et scripts de test sont archivés sur le dépôt GitHub officiel du binôme :

https://github.com/ak4f/tp1-securite-wifi-portail-captif

8. Conclusion et Principes pour l'Ingénieur Logiciel

L'expérimentation réalisée dans ce premier travail pratique démontre sans équivoque la vulnérabilité d'un flux applicatif reposant sur un transport HTTP en clair au sein d'un réseau sans fil partagé. En interceptant directement la charge utile de la trame 11823 sous Wireshark, nous avons constaté qu'aucun algorithme de déchiffrement complexe n'est requis pour compromettre des identifiants non protégés.

Pour l'élève-ingénieur en Génie Logiciel à l'IFRI, trois règles architecturales majeures se dégagent de ces travaux :

1. **Le paradigme du Zero Trust :** Tout réseau de transport (qu'il s'agisse d'un point d'accès public, d'un réseau 4G/5G ou du réseau interne de l'entreprise) doit être traité comme hostile. L'application doit garantir sa confidentialité de bout en bout de manière autonome.

2. **La sécurité par conception (Security by Design) :** Le déploiement de HTTPS avec TLS 1.3 et l'activation de l'en-tête HSTS doivent être intégrés dès la première itération de développement de toute interface exposant des données sensibles.
3. **La complémentarité des couches de défense :** La sécurisation d'un système résulte de la combinaison coordonnée de contrôles réseau (isolation Wi-Fi, segmentation VLAN, WPA3-Enterprise) et de contrôles applicatifs (durcissement de session, objet du TP 2 avec Burp Suite).

Références

[1] IEEE Computer Society, *IEEE Standard for Information Technology : Telecommunications and Information Exchange Between Systems : Local and Metropolitan Area Networks : Specific Requirements : Part 11 : Wireless LAN Medium Access Control (MAC) and Physical Layer (PHY) Specifications*, IEEE Std 802.11-2020.

[2] OWASP Foundation, *OWASP Top 10 :2021 : A02 :2021 Cryptographic Failures*, https://owasp.org/Top10/A02_2021-Cryptographic_Failures/.

[3] E. Rescorla, *The Transport Layer Security (TLS) Protocol Version 1.3*, RFC 8446, Internet Engineering Task Force (IETF), August 2018.

[4] J. Hodges, C. Jackson, A. Barth, *HTTP Strict Transport Security (HSTS)*, RFC 6797, Internet Engineering Task Force (IETF), November 2012.

[5] Gerald Combs et al., *Wireshark Network Protocol Analyzer : User's Guide*, Version 4.x, https://www.wireshark.org/docs/.

[6] Wi-Fi Alliance, *WPA3 Specification : Enhancing Wi-Fi Network Security*, Technical White Paper, 2018.

[7] M. Vanhoef, F. Piessens, *Key Reinstallation Attacks : Forcing Nonce Reuse in WPA2*, Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS '17), pp. 1313-1328, 2017.